Coding-agent exercise reflections
26 October 2026
1. Provenance and materials
These exercises were carried out by the ongoing project coding assistant on 26 October 2026 in a disposable copy of the project. The assistant had access to the Lab 4 instructions, the codebase and tests. It had previously seen the implementation in the conversation, so this was not a blind or fresh-session assessment. The exact model identifier is not asserted. The operational instruction brief is saved in ai-exercises/INSTRUCTIONS.md; it is not presented as a separately typed student message or an invented chat transcript.
2. Exercise 3.3.2 - regenerate the control class
After initial controller tests passed, the entire RecommendationController class was removed from the scratch module. Its supporting helpers and the frozen controller tests were supplied. Test discovery then failed with ImportError, confirming that the class was absent. The working implementation was never deleted from the active project.
Prompting and iteration
The instruction specified the constructor and recommend(request) API, separate forecast/current rankings, eligibility rules, audit behavior and a prohibition on changing tests or calling real services. The first generated replacement preserved the tested business outcomes but the wider suite reported seven errors: scoring_weights and fallback_scoring_policy response metadata had been omitted. Feedback from these actual failures led to a second iteration adding the two fields; the unchanged 150-method regression suite then passed.
Outcome and reflection
The regeneration met the observable test contract after one feedback iteration. It showed why testing only the final winner is insufficient: response metadata is also consumed by callers. Frozen tests made that omission visible. Passing this suite does not establish full behavioral equivalence or show that an agent with no prior context would succeed. Future exercises could use a fresh context and additional independently designed holdout cases. The regenerated class remains experiment evidence; the mechanically refactored working implementation was retained.
3. Exercise 3.3.3 - refactor and regression-check
The further experiment was a focused refactoring review. The recommendation flow was implemented as a module function while the design described a recommendation control class. The assistant was instructed to move this coordination into one small RecommendationController, retain recommend(...) as a compatibility entry point, and leave collection, tariffs, routes and scoring in their existing helpers. The complete current code and regression tests were available; no new framework or service process was requested.
Outcome and reflection
The controller now owns the existing orchestration and has documented constructor/method contracts. The HTTP route and older tests still use the compatibility function, which delegates to it. Current regression passed 151 backend methods, including the later request-boundary test. Sixteen controller cases plus eighteen request boundaries are documented alongside supporting scoring/path cases. Reconciliation also revealed an independent configuration integration defect: the new operator environment names were missing from the loader whitelist.

That was corrected and covered by a regression test. No tariff, model weights or database schema was changed.

Practical assessment

The refactoring improved traceability and made the control-class testing target explicit without adding an ORM, repository hierarchy or dependency container. The cost was one compatibility wrapper and a class that still coordinates a sizeable method; further splitting should be driven by concrete maintenance needs. Automated tests and the Android build are reproducible evidence, while physical-phone acceptance and the team recording remain separate work. The team should review these technical reflections against its own experience before submission.